



Security Audit

PoolOrbit (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	36
Conclusion	37
Our Methodology	38
Disclaimers	40
About Hashlock	41

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The PoolOrbit team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

PoolOrbit is a blockchain-based lottery platform built on Base that enables users to participate in transparent, community-driven prize pools. Participants contribute funds to shared pools, and winners are randomly selected through verifiable on-chain mechanisms. After pools fill up, prizes are instantly distributed from pooled liquidity based on predefined rules.

The platform positions itself as a globally accessible, fair-play alternative to traditional lottery and raffle systems by removing intermediaries and executing all core logic on-chain. This eliminates trust-based outcomes and allows users to track pool activity and results in real time.

At its core, PoolOrbit delivers a modern lottery fully run on the blockchain and combines simplicity, automation, transparency, accessibility, and community participation to create a uniquely gamified financial experience.

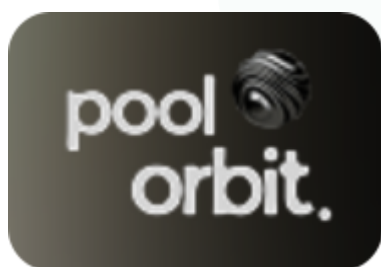
Project Name: PoolOrbit

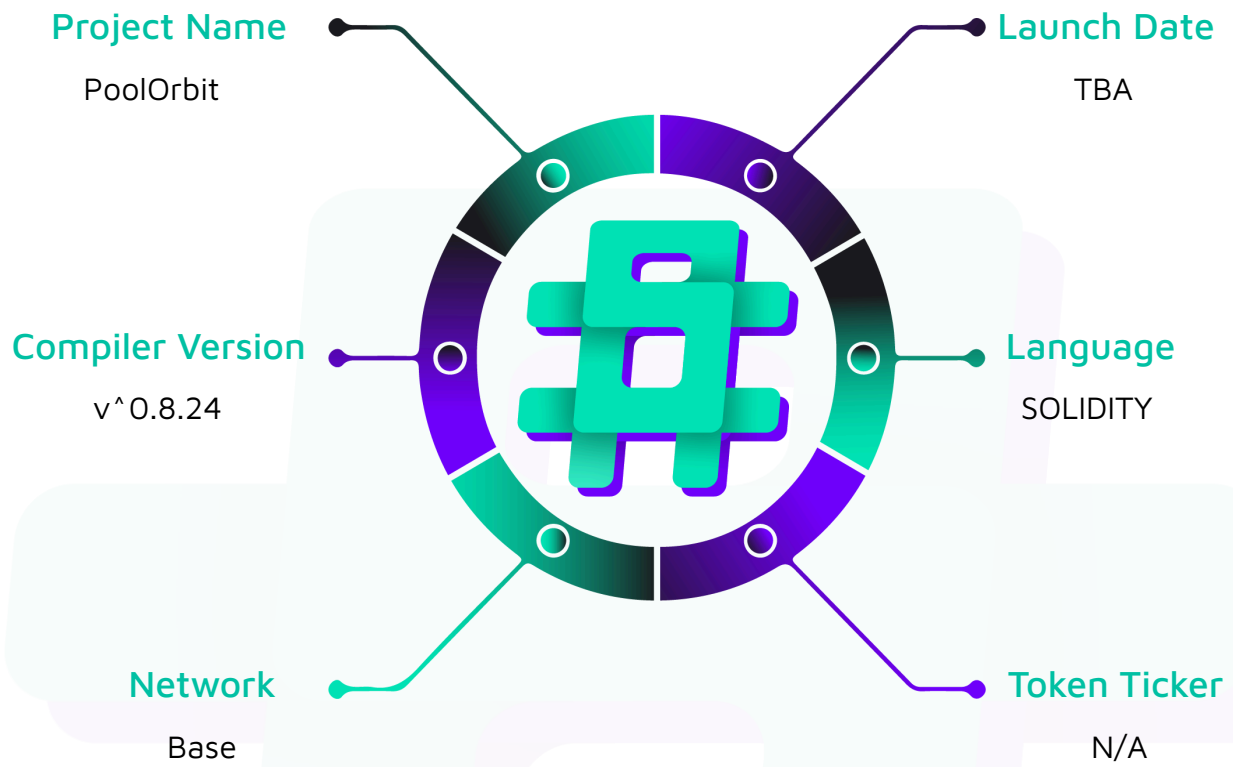
Project Type: DeFi

Compiler Version: ^0.8.24

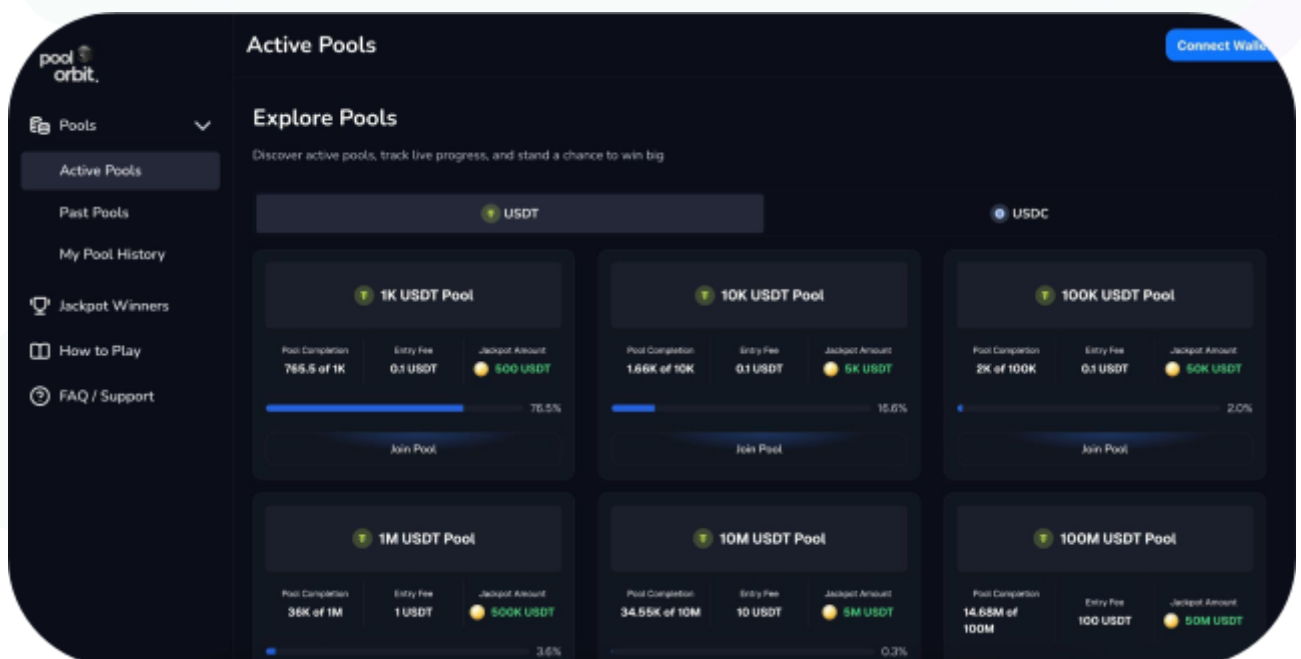
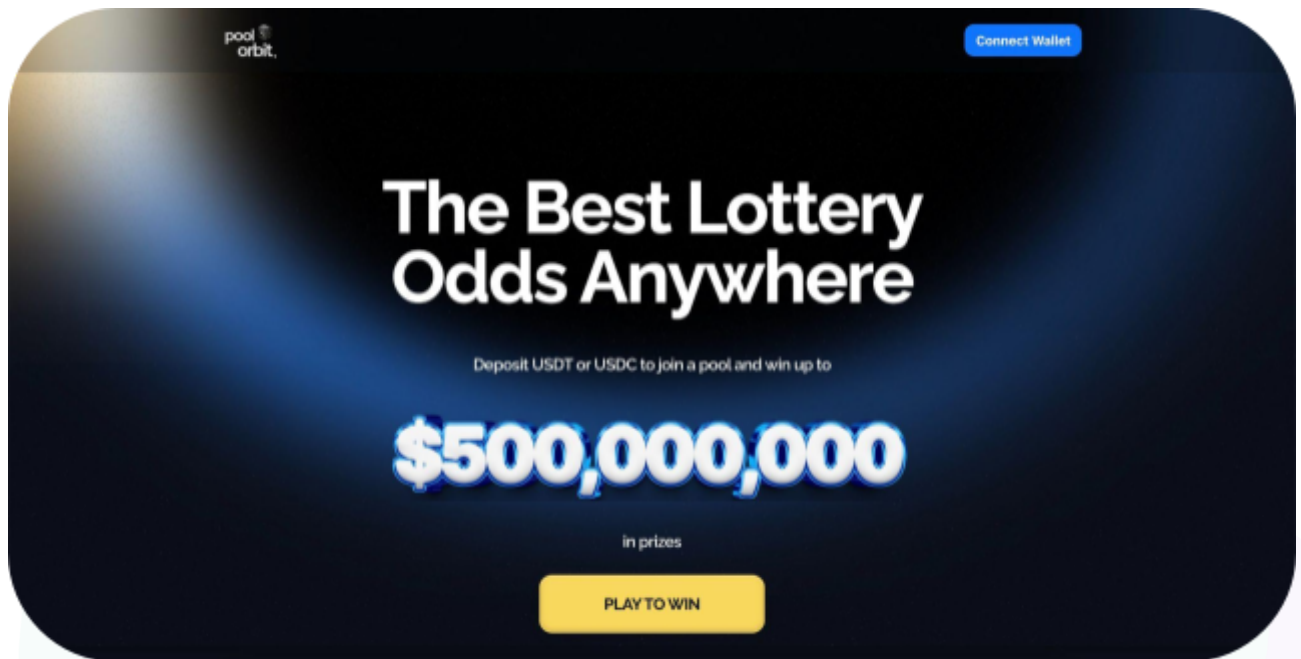
Website: <https://www.poolorbit.com/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the PoolOrbit project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	PoolOrbit Smart Contracts
Network	Ethereum
Language	Solidity
Audit Date	March, 2026
Contract 1	Pool.sol
Contract 2	PoolOrbit.sol
Audited Artifact SHA-256	d16f5dd8e8dd6a119161d494f1b3e0df37aac9abadd7d0fc47953ee7acbc6bb5
Fix Review Artifact SHA-256	d16f5dd8e8dd6a119161d494f1b3e0df37aac9abadd7d0fc47953ee7acbc6bb5

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

4 High severity vulnerabilities

1 Medium severity vulnerability

1 Low severity vulnerability

2 QAs

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
Pool.sol - Allows users to: - Deposit tokens to purchase lottery tickets - Close a full pool to trigger VRF randomness request - Finalize winners via single-call or batch processing - Pay jackpot and consolation winners - Sweep residual balance to treasury after all payouts - Allows the factory to: - Deliver VRF randomness to the pool - Pause, unpause, and force-close the pool	Contract achieves this functionality.
PoolOrbit.sol - Allows the owner to: - Register lottery pool PoolOrbits - Set reward distribution and consolation config - Set treasury address - Configure primary and backup VRF settings - Pause, unpause, and force-close any pool - Allows the VRF coordinator to: - Deliver randomness callback for routing to pools - Allows closed pools to: - Notify the factory to deploy next pool and request VRF	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the PoolOrbit project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the PoolOrbit project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] Pool#batchConsolationPayout - Jackpot winner receives unauthorized consolation prize due to missing ticket exclusion

Description

The `batchConsolationPayout` function is missing a guard to exclude the jackpot ticket from consolation payouts. The single-ticket path (`payConsolationWinner`) has this guard, but the batch path does not. Because `jackpotPaid` and `ticketPaid` are independent storage flags, both the jackpot and consolation payment can execute for the same ticket, resulting in the jackpot winner receiving an extra `consolationPrizeEach` they are ineligible for.

Vulnerability Details

The consolation generation loop in `_processWinners` draws tickets from the full `[1, totalEntries]` range with no exclusion of the jackpot ticket:

```
uint256 ticket = (value % totalEntries) + 1;           // Pool.sol:340 - same range as
jackpot
winningTickets[ticket] = true;                         // Pool.sol:343 - idempotent if
jackpotTicket
consolationWinningTickets.push(ticket);                // Pool.sol:344 - no exclusion check
```

`payConsolationWinner` correctly rejects the jackpot ticket:

```
if (!winningTickets[ticketNumber] || ticketNumber == jackpotTicket) revert
Pool__TicketNotWinning();
```

`payJackpotWinner` sets `jackpotPaid = true` but never sets `ticketPaid[jackpotTicket] = true`. These are independent flags, so after the jackpot is paid, the batch path still finds `ticketPaid[jackpotTicket] == false` and pays consolation. The reverse ordering

also works: batch sets `ticketPayed` but `jackpotPayed` stays false, so `payJackpotWinner` still succeeds.

Proof of Concept

```
it("jackpot winner receives extra consolationPrizeEach via batch path", async () => {
    const { owner, factory, coordinator, alice, bob, usdc } = await
loadFixture(auditFixture);

    // INPUT: 10 USDC pool, 10 entries, 50% consolation = 5 draws

    await factory.connect(owner).setConsolationConfig(5000n);

    const pool = await deployPool(factory, usdc, 1n, ONE("10"), ONE("1"));

    const poolAddr = await pool.getAddress();

    // Alice gets tickets 1-6, Bob gets tickets 7-10

    await usdc.connect(alice).approve(poolAddr, ONE("6"));
    await usdc.connect(bob).approve(poolAddr, ONE("4"));

    await pool.connect(alice).deposit(ONE("6"));
    await pool.connect(bob).deposit(ONE("4"));

    // Craft VRF seeds so jackpotTicket appears in consolationWinningTickets

    const seeds = findCollisionSeeds(10n, 5000n);

    // Close pool -> VRF request -> deliver crafted seeds -> finalize

    const closeTx = await pool.connect(alice).closeIfFull();

    const reqId = await extractRequestId(factory, closeTx);

    await coordinator.fulfillRequest(reqId, seeds);

    await pool.batchFinalizeWinners(5);
```

```

const jackpotTicket = await pool.jackpotTicket();

const jackpotAmount = await pool.jackpotAmount();

const consolationPrize = await pool.consolationPrizeEach();

// VERIFY: jackpotTicket IS in consolationWinningTickets (the collision)

const consolationLen = await pool.generatedConsolationWinners();

let collisionFound = false;

for (let i = 0; i < consolationLen; i++) {

    if ((await pool.consolationWinningTickets(i)) === jackpotTicket) { collisionFound =
true; break; }

}

expect(collisionFound).to.be.true;

// Resolve who owns the jackpot ticket

const entry0 = await pool.getTicketEntry(0);

const entry1 = await pool.getTicketEntry(1);

const jackpotOwnerAddr = Number(jackpotTicket) <= Number(entry0[2]) ? entry0[0] :
entry1[0];

const jackpotSigner = jackpotOwnerAddr === alice.address ? alice : bob;

const otherSigner = jackpotOwnerAddr === alice.address ? bob : alice;

const balBefore = await usdc.balanceOf(jackpotSigner.address);

// EXPLOIT: batch has no ticketNumber != jackpotTicket guard (Pool.sol:220)

// -> pays consolationPrizeEach to jackpot owner on their jackpot ticket

await pool.connect(otherSigner).batchConsolationPayout(10);

```

```

    // Then jackpot pays separately - jackpotPaid is independent of ticketPaid
(Pool.sol:71 vs 85)

    await pool.connect(otherSigner).payJackpotWinner();

    const balAfter = await usdc.balanceOf(jackpotSigner.address);

    const totalReceived = balAfter - balBefore;

    // OUTPUT: both independent flags set for the SAME ticket = double payment proven

    expect(await pool.jackpotPaid()).to.be.true;           // set by payJackpotWinner

        expect(await pool.ticketPaid(jackpotTicket)).to.be.true; // set by
batchConsolationPayout

    expect(totalReceived).to.be.gte(jackpotAmount + consolationPrize); // received MORE
than just jackpot

    expect(totalReceived).to.be.gt(jackpotAmount);

    // DISPLACEMENT PROOF: jackpot ticket consumed a consolation slot,
    // so unique consolation-only winners < targetConsolationWinners

    const targetConsolationWinners = (5000n * 10n) / 10000n; // 5

    const allTickets: bigint[] = [];

    for (let i = 0; i < Number(consolationLen); i++) {

        allTickets.push(await pool.consolationWinningTickets(i));

    }

    const uniqueNonJackpot = new Set(allTickets.filter(t => t !==
jackpotTicket).map(String));

    // Jackpot ticket stole a slot - fewer unique consolation-only winners than target

    expect(BigInt(uniqueNonJackpot.size)).to.be.lt(targetConsolationWinners);

});

```


Impact

The jackpot winner receives one extra `consolationPrizeEach` per collision (~1-5% probability per pool round at default config, near-certain at 1M tickets with 10K draws). A player who would have been selected as a unique consolation winner in that slot receives nothing instead – permanently displaced. The contract does not go insolvent, but the distribution is wrong. Net loss per collision = one `consolationPrizeEach` misallocated from a displaced player to the jackpot winner.

Recommendation

```
// Pool.sol:194 - payJackpotWinner: sync the flags

jackpotPaid = true;

ticketPaid[jackpotTicket] = true;


// Pool.sol:220 - batchConsolationPayout: mirror payConsolationWinner's guard

if (ticketNumber == jackpotTicket) continue;

if (!ticketPaid[ticketNumber]) {
```

Status

Resolved

[H-02] Pool#_processWinners - Missing deduplication in consolation draws causes prize dilution and silent fund diversion to treasury

Description

The consolation winner generation loop does not check whether a ticket has already been selected. The same ticket number can appear multiple times in `consolationWinningTickets[]`, inflating the denominator used to compute `consolationPrizeEach`. Since duplicates are skipped at payout time via `ticketPaid`, fewer unique winners are paid than intended. The undistributed consolation balance accumulates in the contract and is swept to the treasury.

Vulnerability Details

In `_processWinners`, every iteration unconditionally pushes and increments:

```
winningTickets[ticket] = true; // Pool.sol:343 - idempotent, hides the collision

consolationWinningTickets.push(ticket); // Pool.sol:344 - duplicate appended

generatedConsolationWinners++; // Pool.sol:345 - counts every push including dupes
```

`consolationPrizeEach` is then computed at Pool.sol:354:

```
consolationPrizeEach = consolationAmount / targetConsolationWinners;
```

`targetConsolationWinners` equals `generatedConsolationWinners` when the loop exits, including duplicates. The per-winner prize is deflated. At payout time, `batchConsolationPayout` checks `!ticketPaid[ticketNumber]` and skips already-paid duplicates. The unspent consolation stays in the contract and flows to the treasury via `sweepResidualToTreasury`.

Expected collisions (birthday paradox): $E[D] \sim k^2/(2n)$. Default config (10K draws from 1M tickets): ~50 duplicates per pool = ~0.5% of consolation systematically redirected from winners to treasury.

Proof of Concept

```

it("consolationWinningTickets contains duplicates, diluting prizes", async () => {
    const { owner, factory, coordinator, alice, bob, usdc } = await
loadFixture(auditFixture);

    // INPUT: 5 entries, 80% consolation = 4 draws from only 5 tickets -> high collision
chance

    await factory.connect(owner).setConsolationConfig(8000n);

    const pool = await deployPool(factory, usdc, 3n, ONE("5"), ONE("1"));

    const poolAddr = await pool.getAddress();

    await usdc.connect(alice).approve(poolAddr, ONE("3"));

    await usdc.connect(bob).approve(poolAddr, ONE("2"));

    await pool.connect(alice).deposit(ONE("3"));

    await pool.connect(bob).deposit(ONE("2"));

    // Craft seeds that produce a consolation-consolation duplicate

    const seeds = findDuplicateSeeds(5n, 8000n);

    const closeTx = await pool.connect(alice).closeIfFull();

    const reqId = await extractRequestId(factory, closeTx);

    await coordinator.fulfillRequest(reqId, seeds);

    await pool.batchFinalizeWinners(10);

    // Read all consolation winning tickets from the array

    const generated = Number(await pool.generatedConsolationWinners());

    const tickets: bigint[] = [];

```



```

for (let i = 0; i < generated; i++) {

    tickets.push(await pool.consolationWinningTickets(i));

}

// OUTPUT: array has duplicates (e.g. [4, 1, 4, 4])

//     -> generatedConsolationWinners counts dups, inflating consolationPrizeEach
denominator

//     -> duplicate slots skipped at payout (ticketPayed), undistributed funds ->
treasury

const unique = new Set(tickets.map(String));

expect(unique.size).to.be.lt(tickets.length); // fewer unique than total = duplicates
exist

});

```

Impact

Each unique consolation winner receives less than their fair share. With 4 slots but only 2 unique winners (as in the PoC), each winner gets $\text{consolationAmount} / 4$ instead of $\text{consolationAmount} / 2$ – a 50% dilution. At default config on large pools the per-round impact is ~0.5%, but it is systematic and guaranteed. Over many pool cycles, the cumulative diversion from winners to treasury is significant. Users have no on-chain mechanism to detect or contest the dilution.

Recommendation

Use the existing winningTickets mapping as a deduplication check. This also prevents jackpot-consolation collisions (H-01):

```

if (!winningTickets[ticket]) {           // rejects duplicates AND jackpotTicket

    winningTickets[ticket] = true;

    consolationWinningTickets.push(ticket);

    generatedConsolationWinners++;

    emit WinnerTicket(ticket, false);

```



```
}
```

Status

Resolved



[H-03] PoolOrbit#handlePoolClosed - No re-request mechanism for VRF randomness causes permanent fund lock

Description

When a pool closes, `handlePoolClosed` submits a VRF randomness request and immediately overwrites `PoolOrbit.currentPool` to a newly deployed pool. If the Chainlink VRF callback never arrives (subscription drained, coordinator outage, gas limit misconfiguration), the closed pool is permanently stuck with `randomSeedCount = 0`. No function in either contract can re-request randomness or extract funds from a pool in this state.

Vulnerability Details

The close path commits `closed = true` (Pool.sol:309) before calling `handlePoolClosed` (Pool.sol:316). Inside `handlePoolClosed`, `_deployPool` overwrites `PoolOrbit.currentPool` to the new pool (PoolOrbit.sol:225) before `_requestRandomness` is called (PoolOrbit.sol:230).

If the VRF request is accepted by the coordinator but the callback never arrives, the pool enters a terminal stuck state.

Proof of Concept

```
it("all recovery paths revert after close without VRF delivery", async () => {
    const { owner, factory, coordinator, alice, bob, usdc } = await
loadFixture(auditFixture);

    // INPUT: 10 USDC pool, filled by alice+bob

    const pool = await deployPool(factory, usdc, 4n, ONE("10"), ONE("1"));

    const poolAddr = await pool.getAddress();

    await usdc.connect(alice).approve(poolAddr, ONE("6"));

    await usdc.connect(bob).approve(poolAddr, ONE("4"));
```

```

await pool.connect(alice).deposit(ONE("6"));

await pool.connect(bob).deposit(ONE("4"));

// Close pool - VRF request submitted but we NEVER call coordinator.fulfillRequest
await pool.connect(alice).closeIfFull();

// STATE: closed=true, randomSeedCount=0, balance=10 USDC

expect(await pool.closed()).to.be.true;

expect(await pool.randomSeedCount()).to.equal(0);

expect(await usdc.balanceOf(poolAddr)).to.equal(ONE("10"));

// EXPLOIT: every possible recovery path is dead

// 1. Re-close -> closed=true blocks it

    await expect(pool.connect(alice).closeIfFull()).to.be.revertedWithCustomError(pool,
"Pool__Closed");

// 2. Finalize -> randomSeedCount=0 blocks it

    await expect(pool.connect(alice).finalizeWinners()).to.be.revertedWithCustomError(pool,
"Pool__RandomnessMissing");

                                                                    await
expect(pool.connect(alice).batchFinalizeWinners(1)).to.be.revertedWithCustomError(pool,
"Pool__RandomnessMissing");

// 3. Pay jackpot -> winnersFinalized=false blocks it

                                                                    await
expect(pool.connect(alice).payJackpotWinner()).to.be.revertedWithCustomError(pool,
"Pool__WinnersPending");

// 4. Pay consolation -> winnersFinalized=false blocks it

                                                                    await
expect(pool.connect(alice).payConsolationWinner(1)).to.be.revertedWithCustomError(pool,
"Pool__WinnersPending");

```

```

// 5. Sweep -> winnersFinalized=false blocks it

                                                                    await
expect(pool.connect(alice).sweepResidualToTreasury()).to.be.revertedWithCustomError(pool,
"Pool__WinnersPending");

// 6. Deposit more -> closed=true blocks it

await usdc.connect(alice).approve(poolAddr, ONE("1"));

await expect(pool.connect(alice).deposit(ONE("1"))).to.be.revertedWithCustomError(pool,
"Pool__Closed");

// 7. Owner inject randomness -> onlyFactory blocks it (owner != factory)

                                                                    await
expect(pool.connect(owner).receiveRandomness([1n,
2n])).to.be.revertedWithCustomError(pool, "Pool__OnlyFactory");

// 8. forceClosePool -> currentPool is the NEW pool, not this stuck one

                                                                    await
expect(factory.connect(owner).forceClosePool(poolAddr)).to.be.revertedWithCustomError(fac
tory, "PoolFactory__OnlyPool");

// OUTPUT: 10 USDC permanently locked, zero recovery paths

expect(await usdc.balanceOf(poolAddr)).to.equal(ONE("10"));

});

```

Impact

100% of deposited funds are permanently locked with no on-chain recovery path. The amount at risk equals the full `poolSize` of the affected pool. This can be triggered by Chainlink subscription depletion, coordinator outage, or `callbackGasLimit` misconfiguration – conditions outside the protocol's control.

Recommendation

Add an owner-callable re-request function to PoolOrbit:

```

function reRequestRandomness(address pool) external onlyFactoryOwner {

    if (!knownPools[pool]) revert PoolFactory__UnknownPool();

```



```
require(Pool(pool).closed(), "not closed");  
  
require(Pool(pool).randomSeedCount() == 0, "already seeded");  
  
_requestRandomness(pool);  
  
}
```

Status

Resolved

[H-04] Pool#batchConsolationPayout - Blocklisted token recipient permanently bricks batch cursor and blocks treasury sweep

Description

If a consolation winner's address is on the token's blocklist, the `safeTransfer` in `batchConsolationPayout` reverts. Because the cursor increment and the transfer are in the same atomic transaction, the cursor is rolled back and permanently stuck at the blocklisted entry. `sweepResidualToTreasury` requires full cursor exhaustion, so the sweep is also permanently blocked.

Vulnerability Details

In `batchConsolationPayout` (Pool.sol:216-226), the cursor advance and transfer are in the same transaction:

```
consolationPayoutIndex++;                                // Pool.sol:218 -
state change

if (!ticketPayed[ticketNumber]) {
    ticketPayed[ticketNumber] = true;                    // Pool.sol:222 -
state change

    token.safeTransfer(ownerOfTicket, consolationPrizeEach); // Pool.sol:224 -
REVERTS
}
```

If `safeTransfer` reverts, the entire transaction reverts atomically. `consolationPayoutIndex++` is rolled back. Every future call to `batchConsolationPayout` reads the same blocklisted entry at the same index.

`payConsolationWinner` (Pool.sol:199-210) can pay other non-blocklisted winners individually, but it does not reference `consolationPayoutIndex` at all – the batch cursor stays stuck.

`sweepResidualToTreasury` (Pool.sol:232) checks `consolationPayoutIndex < consolationWinningTickets.length` – permanently true while the cursor is stuck, so sweep is permanently blocked.

Proof of Concept

```

it("blocklisted consolation winner permanently sticks batch cursor and blocks sweep",
  async () => {
    const { owner, factory, coordinator, alice, bob, token } = await
loadFixture(blocklistFixture);

    // INPUT: 2 entries, 1 consolation winner, using blocklist-capable token

    await factory.connect(owner).setConsolationConfig(5000n);

    const pool = await deployPool(factory, token, 5n, ONE("2"), ONE("1"));
    const poolAddr = await pool.getAddress();

    await token.connect(alice).approve(poolAddr, ONE("1"));
    await token.connect(bob).approve(poolAddr, ONE("1"));
    await pool.connect(alice).deposit(ONE("1"));
    await pool.connect(bob).deposit(ONE("1"));

    const closeTx = await pool.connect(alice).closeIfFull();
    const reqId = await extractRequestId(factory, closeTx);
    await coordinator.fulfillRequest(reqId, []);
    await pool.batchFinalizeWinners(10);

    // Find out who the consolation winner is, then blocklist them

    const consolationTicket = await pool.consolationWinningTickets(0);
    const entry0 = await pool.getTicketEntry(0);
    const entry1 = await pool.getTicketEntry(1);

    const consolationOwnerAddr = Number(consolationTicket) <= Number(entry0[2]) ? entry0[0]
: entry1[0];

```

```

// EXPLOIT: blacklist the winner AFTER finalization (simulates OFAC sanction)
await token.setBlocklisted(consolationOwnerAddr, true);

// batchConsolationPayout reverts – safeTransfer fails on blocklisted address
// -> consolationPayoutIndex++ at Pool.sol:218 is ROLLED BACK (atomic revert)
await expect(pool.connect(alice).batchConsolationPayout(1)).to.be.reverted;

// OUTPUT: cursor stuck at 0 – did NOT advance past the blocklisted entry
expect(await pool.consolationPayoutIndex()).to.equal(0);

// Pay jackpot if winner is different from blocklisted consolation winner
const jackpotTicket = await pool.jackpotTicket();

const jackpotOwnerAddr = Number(jackpotTicket) <= Number(entry0[2]) ? entry0[0] :
entry1[0];

if (jackpotOwnerAddr !== consolationOwnerAddr) {
    await pool.payJackpotWinner();
}

// sweepResidualToTreasury BLOCKED: consolationPayoutIndex(0) < length(1) at
Pool.sol:232
await expect(pool.sweepResidualToTreasury()).to.be.revertedWithCustomError(pool,
"Pool__InvalidAmount");
});

```

Impact

The blocklisted winner's `consolationPrizeEach` are permanently locked in the contract. Individual `payConsolationWinner` still works for non-blocklisted winners, but the sweep,



which recovers all remaining funds – is permanently blocked. An analogous scenario exists for `payJackpotWinner` (Pool.sol:194): `jackpotPaid = true` is rolled back on revert, independently blocking sweep via the `!jackpotPaid && jackpotAmount > 0` gate.

Recommendation

Use try/catch in the batch loop so the cursor advances regardless of transfer outcome:

```
ticketPaid[ticketNumber] = true;

try token.transfer(ownerOfTicket, consolationPrizeEach) {} catch {

    // route to escrow or emit FailedPayout event
}

// Alternatively: admin cursor override function, or pull-payment pattern where winners
claim to a self-specified address.
```

Status

Resolved

Medium

[M-01] Pool#finalizeWinners - Guaranteed out-of-gas for production-scale pools due to unbounded iteration

Description

`finalizeWinners()` passes the full `totalConsolationWinners` count as the iterations argument to `_processWinners`, attempting to generate all consolation winners in a single transaction. On-chain gas measurement shows ~47K gas per consolation winner, meaning any pool with more than ~636 consolation winners exceeds the 30M block gas limit. The PoC demonstrates a natural OOG failure on 799 winners. The batch alternative `batchFinalizeWinners(N)` succeeds on the same pool.

Vulnerability Details

```
function finalizeWinners() external whenNotPaused {
    uint256 totalConsolationWinners = (consolationWinnerBps * totalEntries) /
    BPS_DENOMINATOR;

    _processWinners(totalConsolationWinners);    // Pool.sol:180 - passes FULL count
}
```

With default `consolationWinnerBps=100` (1%) and 100K entries: 1,000 winners x 50K gas = 50M gas, exceeding the 30M block gas limit by 1.67x.

Proof of Concept

```
it("finalizeWinners fails on 799 winners while batchFinalizeWinners succeeds", async ()
=> {
    const { owner, factory, coordinator, alice, usdc } = await loadFixture(auditFixture);

    // INPUT: 800 entries, consolationWinnerBps=10000 (100%, capped to 799 at Pool.sol:334)
    // 799 winners x ~47K gas/winner = ~37.5M gas > 30M block gas limit

    await factory.connect(owner).setConsolationConfig(10000n);
```

```

const pool = await deployPool(factory, usdc, 6n, ONE("800"), ONE("1"));

const poolAddr = await pool.getAddress();

// Single deposit fills entire pool - 800 entries in one call

await usdc.connect(alice).approve(poolAddr, ONE("800"));

await pool.connect(alice).deposit(ONE("800"));

const closeTx = await pool.connect(alice).closeIfFull();

const reqId = await extractRequestId(factory, closeTx);

await coordinator.fulfillRequest(reqId, []);

// PROOF 1: finalizeWinners FAILS - gas exceeds Hardhat's 30M block gas limit naturally
//   Pool.sol:179 computes totalConsolationWinners = 799
//   Pool.sol:180 passes all 799 to _processWinners in one call
//   The VM cannot execute it within the block gas limit - no artificial cap needed

let finalizeFailed = false;

try {
  await pool.finalizeWinners();
} catch {
  finalizeFailed = true;
}

expect(finalizeFailed).to.be.true;

expect(await pool.winnersFinalized()).to.be.false; // nothing happened

// PROOF 2: batchFinalizeWinners SUCCEEDS on the same pool, same 799 winners
//   Pool.sol:183 lets caller control iterations - 100 per call stays within gas budget

```

```
for (let i = 0; i < 8; i++) {  
    await pool.batchFinalizeWinners(100);  
}  
  
expect(await pool.winnersFinalized()).to.be.true;  
});
```

Impact

`finalizeWinners()` silently reverts with out-of-gas for any pool with >600 consolation winners. No funds are at risk since `batchFinalizeWinners` is a working alternative, but the broken convenience function creates a false expectation that single-call finalization is supported.

Recommendation

Remove `finalizeWinners()` or add a gas-safe threshold check that reverts with a descriptive error. Document `batchFinalizeWinners` as the required finalization path.

Status

Resolved

Low

[L-01] PoolOrbit#getActivePoolsByToken - Unbounded iteration over ever-growing array causes view DoS

Description

PoolOrbit.sol:260-282 iterates all PoolOrbitIds[] in two passes. This array grows with every registerPoolOrbit call and is never pruned. At scale (10K+ PoolOrbits), view calls exceed eth_call gas limits on default node configurations.

Vulnerability Details

The getActivePoolsByToken function performs two full passes over the PoolOrbitIds[] array – first to count matching entries, then to populate the return array. Since PoolOrbitIds[] is append-only (registerPoolOrbit pushes but nothing removes), the iteration cost grows linearly with total PoolOrbits ever registered, regardless of how many are active or match the queried token.

Impact

Off-chain consumers (frontends, indexers) calling getActivePoolsByToken will receive timeout or gas-limit errors once PoolOrbit count reaches ~10K+. This does not affect on-chain state or funds, but breaks read-path availability for any integration relying on this view function.

Recommendation

Per-token index mapping or paginated query.

Status

Resolved

QA

[Q-01] Pool#sweepResidualToTreasury - Operator precedence ambiguity in compound boolean condition

Description

Pool.sol:232 relies on implicit `&& > ||` precedence. Logic is correct but missing explicit parentheses is a readability and maintenance hazard.

Vulnerability Details

The compound boolean condition in `sweepResidualToTreasury` mixes `&&` and `||` operators without explicit parentheses. While Solidity's operator precedence rules produce the intended evaluation order, the lack of parentheses makes the intended grouping ambiguous to human readers and increases the risk of introducing logic errors during future modifications.

Impact

No functional impact

Recommendation

Add explicit parentheses to clarify the intended boolean grouping.

Status

Resolved

[Q-02] Pool#consolationAmount - Public state variable becomes stale after finalization

Description

Pool.sol:74 `consolationAmount` is public but never updated as prizes are claimed. External integrations reading it as "remaining unclaimed" will get incorrect values. No internal logic is affected.

Vulnerability Details

`consolationAmount` is set once during finalization and remains at its initial value as consolation prizes are paid out. Any external contract or frontend querying this value after partial payouts will read the original total rather than the remaining unclaimed amount, leading to incorrect display or accounting in downstream integrations.

Impact

No impact on internal contract logic or fund safety. External readers may display misleading "remaining consolation" values.

Recommendation

Zero the value at finalization or add a dynamic view function that computes the actual remaining unclaimed consolation amount.

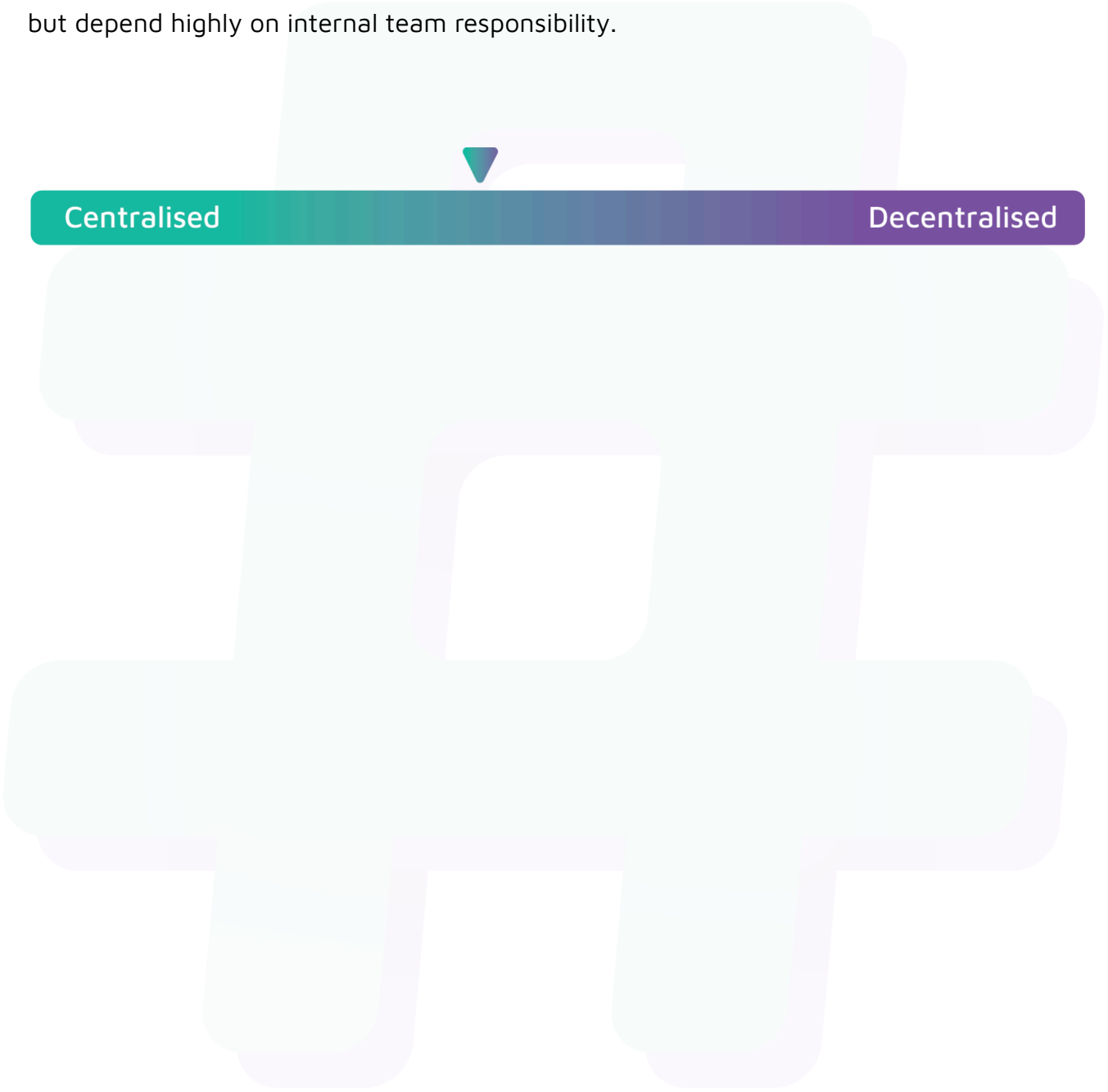
Status

Acknowledged

Centralisation

The PoolOrbit project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the PoolOrbit project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.